

BigCodeBench/737 Evaluations

Task Description

Calculate the median of all elements in a nested list 'L'.

- Output: median (float)
 - Required imports: numpy as np, math
-

Model 1 Evaluation

model1_efficiency

Total steps: 3+ (reasoning content appears incomplete/cut off)

- Step 1: Problem understanding - **Essential**
- Step 2: Considered different flattening approaches, chose iterative - **Essential**
- Step 3: Describes iterative flattening mechanism - **Essential**

Note: The reasoning content is incomplete, cutting off mid-sentence in step 3

model1_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - logically follows from need to flatten nested list
- Step 3: Coherent - explains the chosen iterative approach

Internal Consistency: No contradictions identified in the provided portion

model1_completeness

Problem Coverage:

- Flatten nested list: Partially addressed (incomplete)
- Calculate median: Not addressed (incomplete)
- Return float: Not addressed (incomplete)
- Use required imports: Not addressed (incomplete)

Factual Correctness: What's provided is correct

Constraint and edge case handling: Not addressed due to incomplete reasoning

Model 2 Evaluation

model2_efficiency

Total steps: 5

- Step 1: Problem understanding - **Essential**
- Step 2: Describes recursive flattening approach - **Essential**
- Step 3: Explains median calculation logic - **Essential**
- Step 4: Using np.median and type casting - **Essential**
- Step 5: Wrapping logic in task_func - **Non-essential** (redundant, already implied)

model2_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - follows from need to flatten nested structure
- Step 3: Coherent - follows from having flattened list
- Step 4: Coherent - implementation detail of step 3
- Step 5: Coherent - finalizing the solution

Internal Consistency: No contradictions identified

model2_completeness

Problem Coverage:

- Flatten nested list: Yes
- Calculate median: Yes
- Return float: Yes
- Use required imports: Yes

Factual Correctness: Correct understanding of median calculation and numpy usage

Constraint and edge case handling: Does not address empty list case or non-numeric elements

Model 3 Evaluation

model3_efficiency

Total steps: 6

- Step 1: Problem understanding with edge case consideration - **Essential**

- Step 2: Explains need for flattening and filtering numeric elements - **Essential**
- Step 3: Describes recursive helper function approach - **Essential**
- Step 4: Using np.median for calculation - **Essential**
- Step 5: Handling empty list edge case - **Essential**
- Step 6: Main function orchestration - **Essential** (comprehensive but verbose)

model3_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - elaborates on flattening requirements
- Step 3: Coherent - implementation approach for step 2
- Step 4: Coherent - calculation method
- Step 5: Coherent - necessary edge case handling
- Step 6: Coherent - putting all pieces together

Internal Consistency: No contradictions, maintains consistent approach throughout

model3_completeness

Problem Coverage:

- Flatten nested list: Yes
- Calculate median: Yes
- Return float: Yes
- Use required imports: Yes
- Filter numeric types: Yes (explicitly mentioned)

Factual Correctness: All statements are factually correct, including np.median behavior

Constraint and edge case handling:

- Explicitly handles empty list case (returns np.nan)
- Considers non-numeric elements
- Most comprehensive edge case coverage

Model 4 Evaluation

model4_efficiency

Total steps: 5 (includes code implementation)

- Step 1: Flatten using iterative stack-based approach - **Essential**
- Step 2: Check empty list - **Essential**
- Step 3: Sort for median calculation - **Essential**
- Step 4: Calculate median for odd/even cases - **Essential**
- Step 5: Return as float - **Essential**

model4_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - necessary check before calculation
- Step 3: Coherent - required for median
- Step 4: Coherent - implements median calculation logic
- Step 5: Coherent - ensures correct return type

Internal Consistency: No contradictions, consistent iterative approach

model4_completeness

Problem Coverage:

- Flatten nested list: Yes
- Calculate median: Yes (manual implementation)
- Return float: Yes
- Use required imports: Partially (doesn't use numpy for median calculation)

Factual Correctness: Median calculation logic is correct

Constraint and edge case handling:

- Handles empty list (returns float('nan'))
 - Does not filter for numeric types only
-

Model 5 Evaluation

model5_efficiency

Total steps: 5

- Step 1: Flatten list explanation - **Essential**
- Step 2: Sort elements - **Essential**
- Step 3: Calculate median based on odd/even - **Essential**

- Step 4: Using numpy.median with helper function - **Essential**
- Step 5: Edge case note - **Non-essential** (just acknowledges limitation without solution)

model5_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - required for median
- Step 3: Coherent - explains median logic
- Step 4: Coherent - implementation approach
- Step 5: Coherent - acknowledges limitation

Internal Consistency: No contradictions identified

model5_completeness

Problem Coverage:

- Flatten nested list: Yes
- Calculate median: Yes
- Return float: Yes
- Use required imports: Yes

Factual Correctness: All statements are correct

Constraint and edge case handling: Acknowledges empty list case but doesn't provide solution

Model 6 Evaluation

model6_efficiency

Total steps: 5

- Step 1: Problem understanding - **Essential**
- Step 2: Need to flatten - **Essential**
- Step 3: Median calculation logic - **Essential**
- Step 4: Using np.median - **Essential**
- Step 5: Flattening approach - **Essential**

model6_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - identifies key requirement
- Step 3: Coherent - explains median calculation
- Step 4: Coherent - implementation choice
- Step 5: Coherent - details flattening approach (though could have come earlier)

Internal Consistency: No contradictions identified

model6_completeness

Problem Coverage:

- Flatten nested list: Yes
- Calculate median: Yes
- Return float: Yes
- Use required imports: Yes

Factual Correctness: All statements are correct

Constraint and edge case handling: Not addressed

Summary

- **Most Complete:** Model 3 (addresses edge cases, numeric filtering)
- **Most Efficient:** Models 2, 4, 6 (minimal non-essential steps)
- **Best Logic:** Models 3, 4 (clear, coherent progression)
- **Incomplete:** Model 1 (reasoning cut off)